# Week 5

# Classification

*Can the Movement Data Tell Us*
*What the Clinician Already Knows?*

| Section | Question it answers |
|---|---|
| 1. The Clinical Question | Could we diagnose from EMG alone? (raw vs PCA) |
| 2. Drawing a Line | What is the simplest approach? |
| 3. Testing Honestly | How do we measure real-world accuracy? |
| 4. Reading the Scoreboard | Where are we right and wrong? |
| 5. Can We Do Better? | Which muscles matter most for diagnosis? |
| 6. Feature Representation | Raw EMG vs PCA vs Varimax — does it matter? |
| 7. The Payoff | How close did we get to the clinician? |

# 1. The Clinical Question

In our dataset, we **already know** which subjects are healthy and which are impaired — the clinician told us. But suppose you are handed just the EMG recordings, movement accuracy scores, and kinematic traces, with no clinical labels attached. Could you figure out who is who from the motor data alone?

This is not a hypothetical scenario. In many clinical settings, an expert diagnosis is slow, expensive, or invasive. If muscle activation patterns contain enough information to reliably distinguish healthy from impaired, we could build a **screening tool** — a way to flag subjects who need further evaluation, based purely on their movement data.

Week 4 ended with this figure — all 480 trials projected into a shared PCA space, colored by target direction:

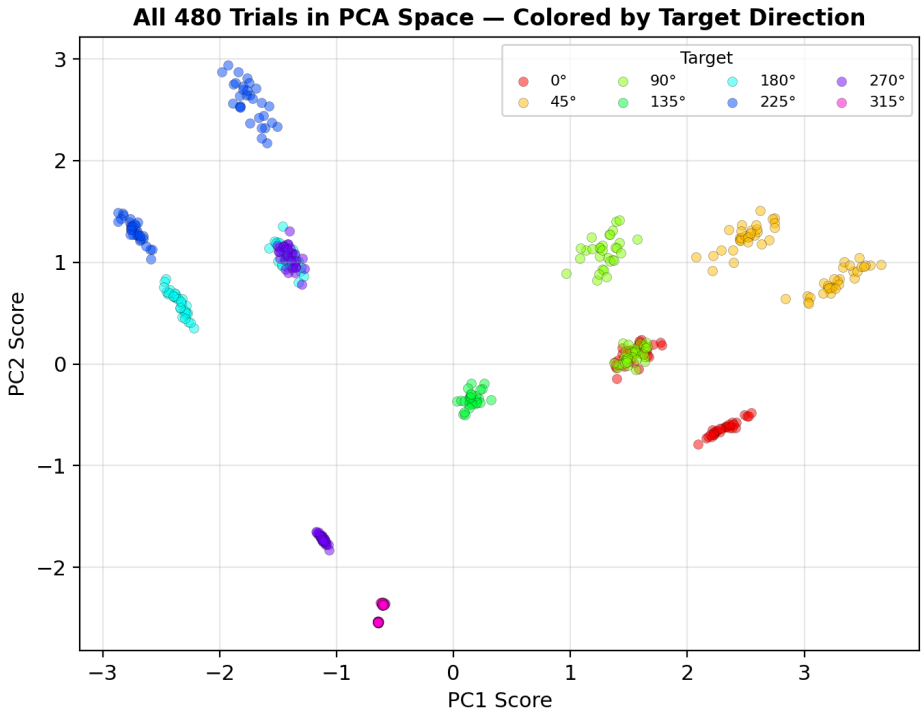


*Figure 1a (recap from Week 4). All 480 trials in PCA space, colored by target direction. Each trial is described by the peak EMG envelope amplitude — the maximum value of the low-pass filtered, full-wave rectified EMG signal — for each of the 6 muscles. These 6 values are standardized and fed into PCA; PC1 and PC2 represent the first two principal components. For several directions, two sub-groups are visible within each cluster.*

We noticed that several directional clusters split into two sub-groups. Now let us reveal what those sub-groups are by re-coloring the **exact same points** using the clinical labels:

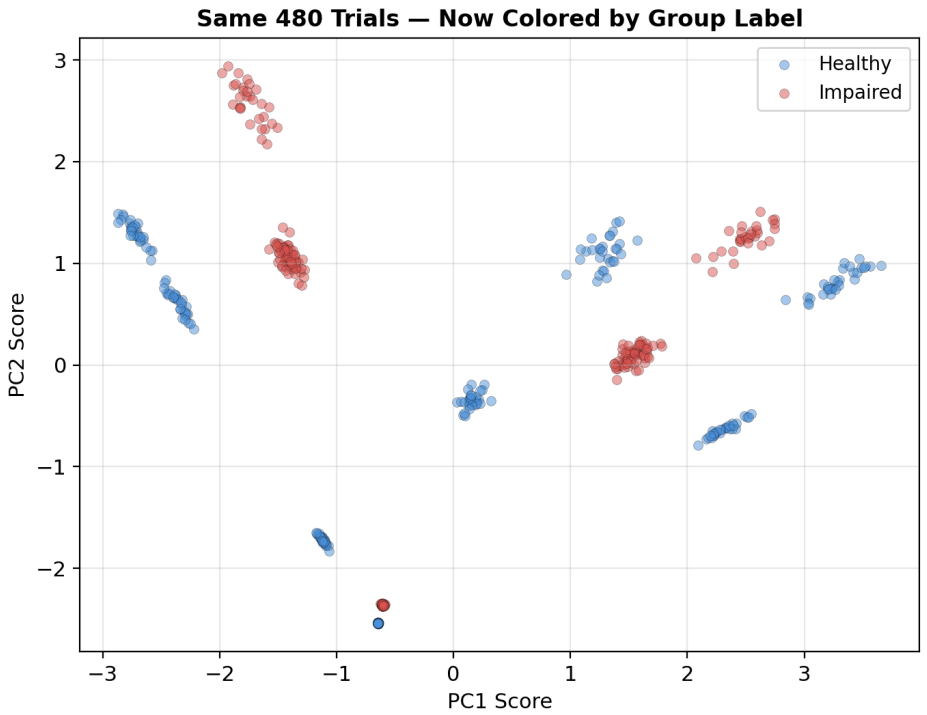


*Figure 1b. The same 480 trials, now colored by clinical group. For each target direction, healthy (blue) and impaired (red) form nearby but distinct sub-clusters. The impaired group is systematically shifted — a signature of altered synergy structure.*

The spatial offset is visible by eye. But is it **reliable enough** to serve as a diagnostic tool? If we gave an algorithm only the EMG features and asked it to guess each trial's group, how often would it get it right? **Classification** is the tool that answers this question.

We will ask two things of our dataset:

- **Direction decoding:** Given only a trial's muscle activations, can we determine which of the 8 targets the subject was reaching toward?
- **Clinical screening:** Given only the EMG, can we distinguish healthy from impaired? How close can we get to the clinician's labels?

And we will track **two feature representations** throughout:

- **Raw EMG** — all 6 peak muscle amplitudes (the full information).
- **PCA scores** — the 2 principal components from Week 4 (a compressed summary).

Running both side by side will tell us whether the PCA compression from Week 4 preserves the diagnostic signal or throws it away.

> The question is simple: the clinician's labels are our ground truth. Classification asks how much of that truth is contained in the movement data alone. We track two representations — raw EMG and PCA — to see which carries more diagnostic information.

## 2. Drawing a Line in the Data

Look at Figure 1b again. The healthy and impaired trials are offset from each other in PCA space. If you had to classify new trials by hand, you would probably draw a line between the two groups and assign everything on one side to 'healthy' and the other to 'impaired.' That intuition is exactly what **logistic regression** does — it finds the best dividing line through the data.

Instead of a hard boundary, logistic regression outputs a **probability** that each trial is impaired:

$$P(\text{impaired} \mid x) = 1 / (1 + e^{-(w \cdot x + b)})$$

The **sigmoid** function squashes any number into [0, 1]. The weights **w** tell us how each muscle contributes: a positive weight on a given muscle means higher activation increases the probability of 'impaired.' Trials above P = 0.5 are classified as impaired; below 0.5 as healthy.

The weights are directly interpretable in motor control terms. If the model assigns a large positive weight to shoulder flexors but near-zero to wrist extensors, it is telling us that shoulder flexor activation is the feature that most distinguishes the two groups — consistent with proximal-muscle impairment patterns.

### Worked example: three real trials

How does the model find the right weights? **Fitting** (or training) means searching for the values of w and b that make the model's predicted probabilities match the clinician's labels as closely as possible. Specifically, the algorithm maximizes the **log-likelihood** — it adjusts weights so that impaired trials get high P(impaired) and healthy trials get low P(impaired). In sklearn, a single call to `model.fit(X, y)` runs this optimization and stores the result in `model.coef_` and `model.intercept_`.

There is one setting we must choose before fitting: the **regularization strength C**. Regularization prevents the model from assigning extreme weights that overfit the training data. A small C means strong regularization (weights kept small); a large C means the model is free to fit the data aggressively. For now we use sklearn's default of `C = 1.0` — a reasonable middle ground. In Section 5, we will search systematically for the best value.

When we fit logistic regression to our PCA scores (2 features) with the binary clinical labels, the fitted weights are $w_1 = -0.016$ (PC1) and $w_2 = 0.158$ (PC2), with intercept $b \approx 0$. Notice that $w_2$ is 10× larger — the model learned that PC2 is the axis that separates the groups, while PC1 (which captures directional variance) is nearly irrelevant for diagnosis. Let us trace three actual trials through the computation:

**Clearly healthy trial** (PC1 = −0.65, PC2 = −2.55): $z = (-0.016)(-0.65) + (0.158)(-2.55) + 0 = -0.39$. Then $\sigma(-0.39) = 1/(1 + e^{0.39}) =$ **0.40**. Since 0.40 < 0.5, predict **healthy**. Correct.

**Borderline trial** (PC1 = +1.66, PC2 = +0.17): $z \approx 0.00$. Then $\sigma(0) = 0.50$ exactly — a coin flip. This trial sits right on the decision boundary.

**Clearly impaired trial** (PC1 = −1.93, PC2 = +2.94): $z = 0.50$. Then $\sigma(0.50) =$ **0.62**. Since 0.62 > 0.5, predict **impaired**. Correct.

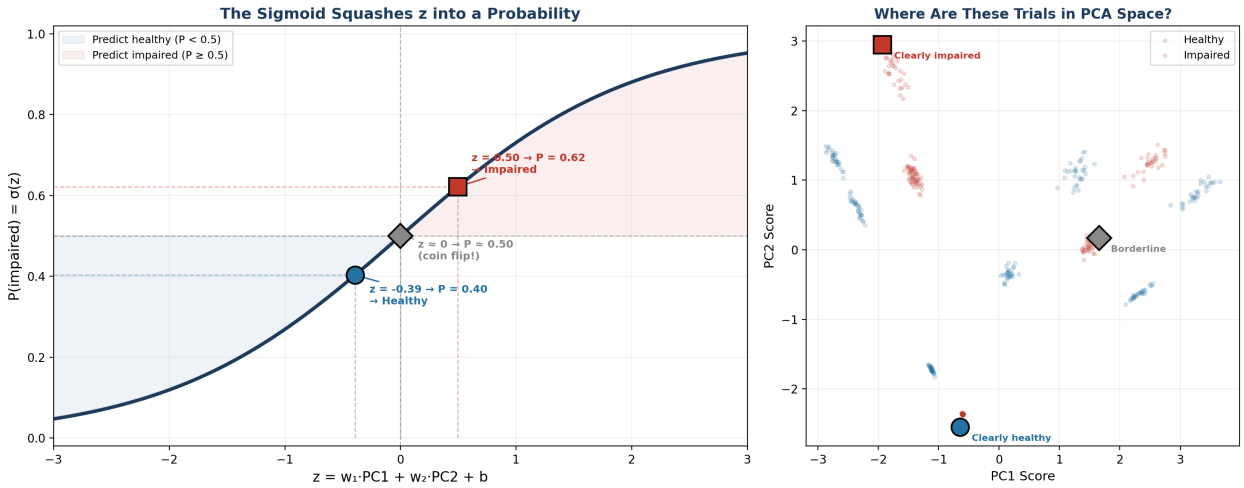


*Figure 2a. Worked example with three real trials. Left: the sigmoid curve with each trial's z-score mapped to its probability. The blue region (P < 0.5) predicts healthy; the red region (P ≥ 0.5) predicts impaired. Right: the same three trials shown in PCA space. The 'clearly healthy' trial has low PC2 (pushed left on the sigmoid); the 'clearly impaired' trial has high PC2 (pushed right).*

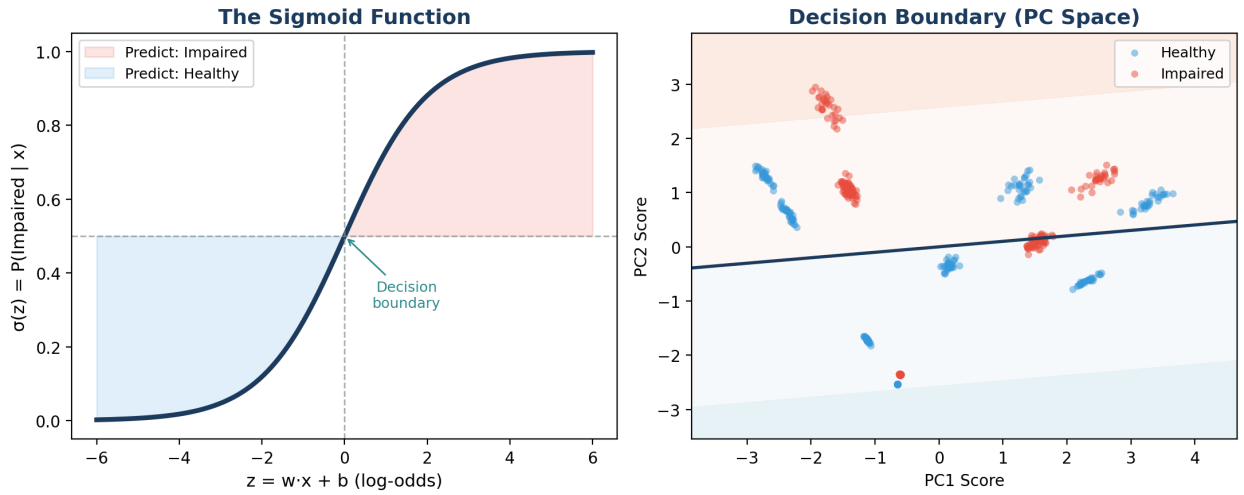


*Figure 2b. Left: the sigmoid maps a linear combination of muscle features to a probability. Right: the resulting decision boundary in PCA space. Blue/red shading shows classifier confidence. The overlap zone contains clinically ambiguous trials.*

## Extending to 8 directions

For the direction-decoding problem, we need 8 classes instead of 2. **Softmax regression** (multinomial logistic regression) generalizes the sigmoid to multiple classes. Each direction gets its own weight vector — effectively its own muscle-pattern template. A new trial is assigned to whichever direction's template it matches best.

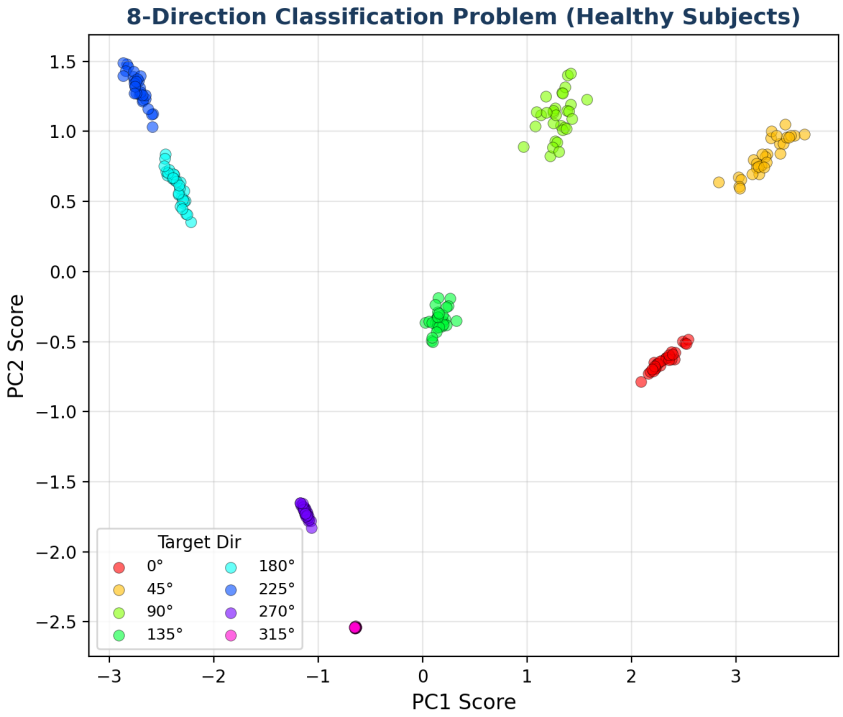


*Figure 3. The 8-direction decoding problem (healthy subjects only) in PCA space. Directions with similar muscle patterns (neighboring angles) overlap more. The classifier must learn to separate all 8 clusters.*

## 3. Testing Honestly

We have trained a classifier. Now the critical question: **how accurate is it, really?** It is tempting to check accuracy on the same data we trained on, but that is like giving a student the exam answers during the test — the score tells us nothing about what they actually learned.

**Cross-validation** solves this by repeatedly holding out a portion of the data, training on the rest, and measuring accuracy on the held-out portion. But for motor control data, *how* we hold out data matters enormously.

### The trap — trial-level splitting

In standard 5-fold cross-validation, trials are randomly assigned to folds. But our dataset has multiple trials per subject. If Subject 3 has trials in both the training and test sets, the classifier can pick up on subject-specific quirks — the unique way Subject 3 co-activates their muscles — rather than learning general features of healthy vs impaired. This inflates accuracy and gives us false confidence.

### The solution — Leave-One-Subject-Out (LOSO)

Hold out **all** trials from one subject, train on everybody else, predict the held-out subject. Repeat for every subject. The model never sees the test subject during training. LOSO accuracy answers the clinically relevant question: *if a brand-new patient walks in, how well can we classify them?*

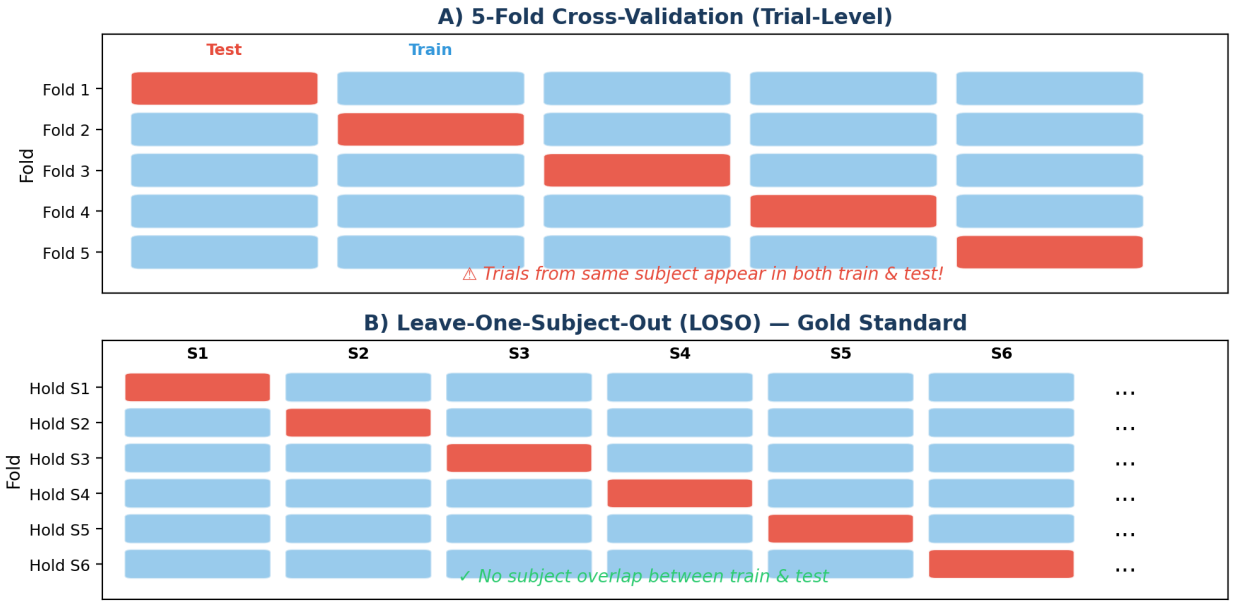


*Figure 4. Two cross-validation strategies. (A) 5-fold CV splits trials randomly — the same subject can appear in both train and test. (B) LOSO holds out an entire subject per fold, guaranteeing no data leakage.*

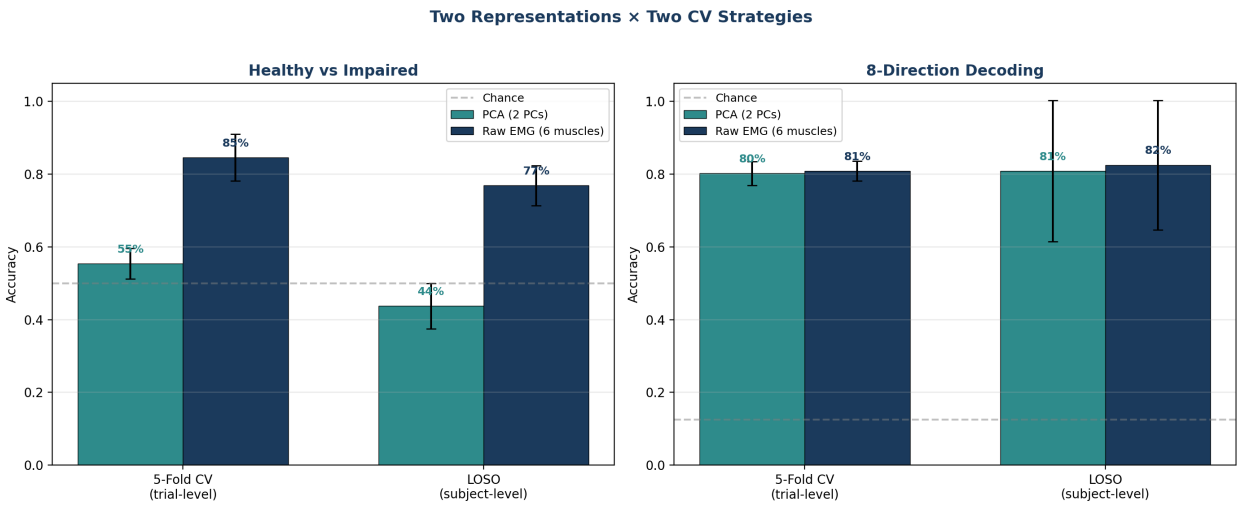


*Figure 5. Cross-validation results for both feature representations. Left (binary): raw EMG (77% LOSO) dramatically outperforms PCA (44% LOSO) for the clinical task — compressing 6 muscles into 2 PCs loses the diagnostic signal. Right (8-direction): both representations perform similarly (~81%) because directional information is captured well by the first 2 PCs. In both panels, the gap between 5-fold and LOSO shows the inflation from data leakage.*

Figure 5 reveals two patterns worth unpacking. First, look at the **5-fold vs LOSO gap**. For the binary task (left panel), trial-level CV gives 85% but LOSO drops to 77% — a large inflation. For 8-direction decoding (right panel), the gap is much smaller. Why? The binary task asks whether a subject is healthy or impaired. Each subject has a stable group identity across all their trials, so when trial-level CV leaks some of Subject 3's trials into training, the model memorizes Subject 3's idiosyncratic muscle pattern and trivially recognizes their held-out trials. LOSO forces the model to generalize across subjects — a much harder task.

Direction decoding is different. Within a single subject, the 8 directions produce genuinely distinct muscle patterns — reaching right activates different muscles than reaching left, regardless of who is doing the reaching. So even when LOSO holds out a new subject, the directional muscle-pattern templates learned from other subjects transfer well. The subject-specific bias matters less because the signal (direction) varies *within* each subject rather than *between* subjects.

Second, look at the **raw EMG vs PCA** comparison. For direction decoding, both representations perform similarly (~81%) — the first 2 PCs capture the directional structure well, as we saw in Week 4. But for the clinical task, PCA collapses to 44% (below chance!). The diagnostic signal — the subtle differences in how healthy and impaired subjects co-activate muscles — lives in the dimensions PCA discarded as 'low variance.' Variance and diagnostic relevance are not the same thing.

Two lessons: (1) always use LOSO when subjects contribute multiple trials. (2) PCA compression that is great for summarizing variance (Week 4) can destroy the signal needed for classification. Raw EMG preserves all 6 muscles — and for clinical diagnosis, those extra dimensions matter.

# 4. Reading the Scoreboard

Overall accuracy tells us *how often* we are right, but not *where* we go wrong. For motor control data, the pattern of errors is often more informative than the accuracy number itself.

## Confusion matrix — which directions confuse each other?

For the 8-direction task, the **confusion matrix** reveals which target directions the classifier mixes up. Rows are the true directions; columns are predictions. Diagonal entries are correct; off-diagonal entries are errors. Neighboring directions (e.g., 0° and 45°) share similar muscle patterns and are confused more often — exactly what muscle synergy theory predicts.

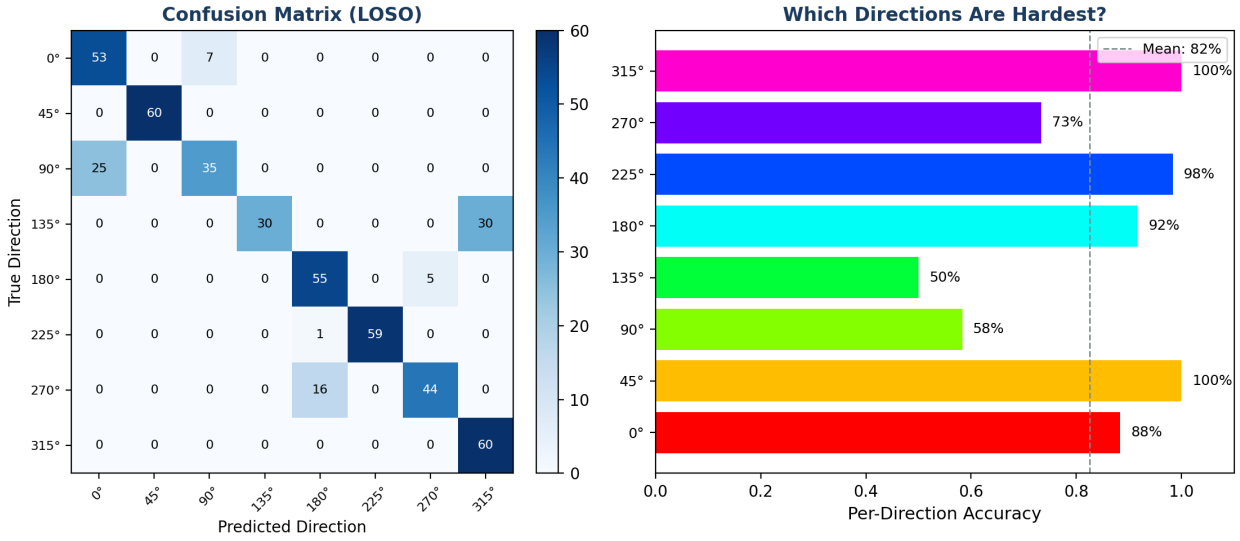


*Figure 6. Left: 8-direction confusion matrix (LOSO, all 480 trials). Some directions are classified perfectly (45°, 180°, 225°, 315°) while others are confused in pairs: 0° with 90°, 135° with 315°, 270° with 180°. These confusions occur between directions whose muscle patterns overlap in PCA space — biomechanically sensible. Right: per-direction accuracy.*

The confusion matrix is the right tool for the direction task because all 8 classes are equally important — misclassifying 90° as 0° is no worse than misclassifying 270° as 180°. But for the clinical task, the stakes are asymmetric. Missing an impaired patient (a **false negative**) means they go undiagnosed; flagging a healthy person as impaired (a **false positive**) means an unnecessary follow-up. These two error types have very different costs, and a single accuracy number hides which one we are making.

## From hard labels to soft probabilities

There is a hidden step in the confusion matrix that we glossed over. To fill in each cell, we need a hard prediction for every trial: 'healthy' or 'impaired.' But logistic regression does not directly output a label — it outputs a **probability**. For each trial, the model says something like 'this trial has a 0.73 probability of being impaired.' To get a label, we apply a **threshold**: if P ≥ 0.5, call it impaired; otherwise, healthy. The confusion matrix above used that default threshold of 0.5 without saying so.

But why 0.5? There is nothing sacred about it. If the cost of missing an impaired patient is high (say, a child who needs early intervention), we might lower the threshold to 0.3 — catching more true cases at the price of more false alarms. If the follow-up test is invasive or expensive, we might raise it to 0.7 — only flagging patients we are quite confident about. Each threshold gives a different confusion matrix with a different balance of errors.

The **ROC curve** is the tool that maps out this entire landscape. Instead of reporting one confusion matrix at one threshold, it shows what happens at *every* threshold simultaneously.

Figure 6b makes the threshold concept visual. Each panel shows the distribution of the model's predicted probabilities for healthy trials (blue) and impaired trials (red). A good classifier pushes these two distributions apart; the overlap region is where errors live. The dashed line is the threshold — everything to its right gets labeled 'impaired.'

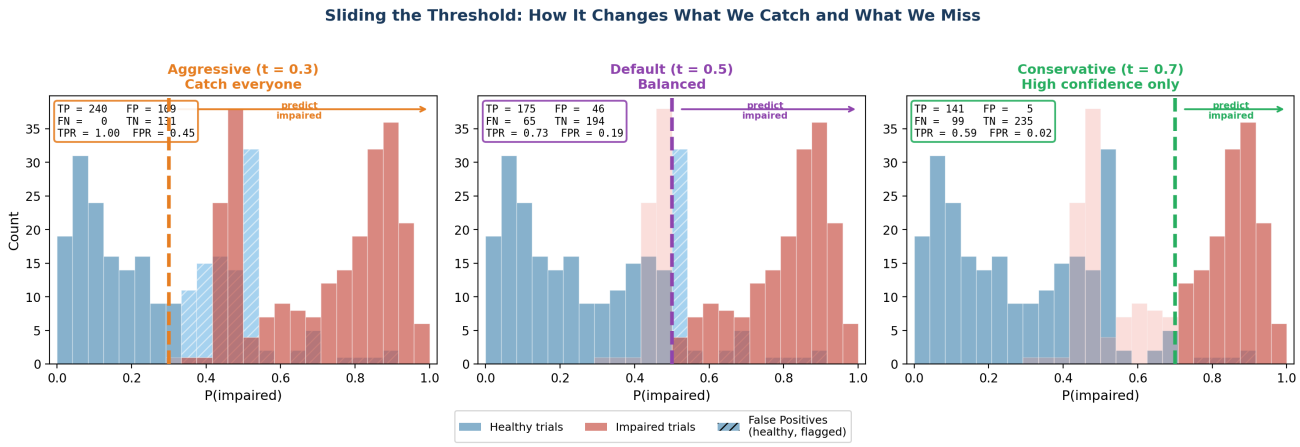


*Figure 6b. The threshold determines the error balance. Each panel shows the same two distributions of predicted P(impaired) for healthy (blue) and impaired (red) trials. Moving the threshold left (t = 0.3) catches all impaired trials (TP = 240) but sweeps in 109 healthy trials as false positives. Moving it right (t = 0.7) nearly eliminates false positives (FP = 5) but misses 99 impaired trials. The hatched region shows healthy trials incorrectly classified as impaired (false positives).*

Look at the overlap zone around P = 0.4–0.6 in Figure 6b. This is where the two groups' muscle patterns are most similar — the clinically ambiguous cases. No threshold can cleanly separate this region. The question is which errors you prefer: missing impaired patients (move threshold right) or over-flagging healthy ones (move threshold left).

## ROC curve — sensitivity vs false alarms

At each threshold we can compute two numbers from the resulting confusion matrix: the **true positive rate** (TPR, or sensitivity — what fraction of truly impaired trials did we catch?) and the **false positive rate** (FPR — what fraction of truly healthy trials did we incorrectly flag?). The ROC curve plots TPR against FPR as the threshold sweeps from 0 to 1.

## Worked example: three thresholds on our data

Our raw-EMG model assigns each of the 480 trials a probability of being impaired. There are 240 truly healthy and 240 truly impaired trials. Let us compute TPR and FPR at three thresholds:

**Threshold = 0.3** (aggressive — flag almost everyone): 240 of 240 impaired are caught (TPR = 240/240 = **1.00**), but 109 of 240 healthy are also flagged (FPR = 109/240 = **0.45**). We miss nobody, but nearly half the healthy patients get unnecessary follow-ups. ROC point: (0.45, 1.00).

**Threshold = 0.5** (default): 175 of 240 impaired are caught (TPR = 175/240 = **0.73**), and 46 of 240 healthy are flagged (FPR = 46/240 = **0.19**). A reasonable middle ground. ROC point: (0.19, 0.73).

**Threshold = 0.7** (conservative — only flag high confidence): 141 of 240 impaired are caught (TPR = 141/240 = **0.59**), and only 5 of 240 healthy are flagged (FPR = 5/240 = **0.02**). Very few false alarms, but we miss 41% of impaired patients. ROC point: (0.02, 0.59).

Each threshold gives one (FPR, TPR) point. The **ROC curve** connects all such points as the threshold sweeps from 0 to 1. A perfect classifier would hug the top-left corner (TPR = 1, FPR = 0 at some threshold); random guessing traces the diagonal. The **area under the curve (AUC)** summarizes this into a single number: 0.5 = guessing, 1.0 = perfect separation.

In a clinical screening context, this tradeoff is concrete. A tool that misses 30% of impaired patients (low sensitivity) is dangerous. One that sends 40% of healthy patients for unnecessary follow-up (low specificity) is costly. The ROC curve lets the clinician choose where to set the dial.

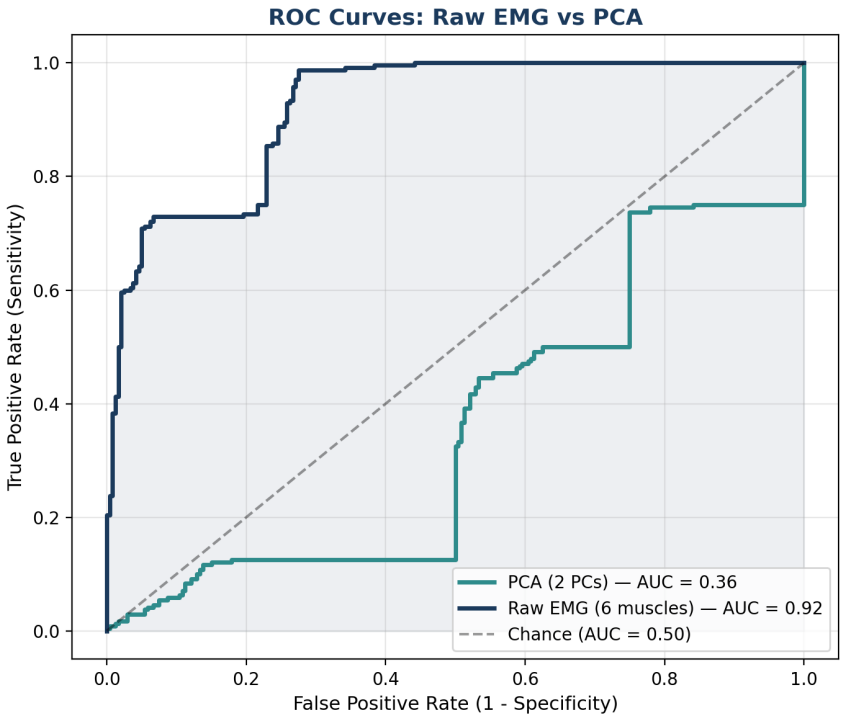


*Figure 7. ROC curves for healthy vs impaired (LOSO, C = 1.0), comparing both representations. Raw EMG (AUC = 0.92) shows strong discriminative ability — the classifier is a useful screening tool. PCA with 2 components (AUC = 0.36) performs worse than chance, confirming that the diagnostic signal lives in the dimensions PCA discarded. After tuning C in Section 5, raw EMG AUC improves to 0.94 (Figure 13).*

## Thought exercise: what does AUC = 0.36 actually mean?

An AUC below 0.5 is startling — it means the PCA classifier is *worse* than flipping a coin. How is that possible? If the model had no information at all, it would guess randomly and land at AUC = 0.50. To get *below* 0.50, the model must be systematically getting things backwards — assigning higher P(impaired) to healthy trials and lower P(impaired) to impaired trials. In other words, the 2-PC representation is not just uninformative; it is actively **misleading** the classifier about group membership.

This raises a natural question: *would adding a third PC fix the problem?* After all, PCA with 2 components captures about 75% of the total variance. Perhaps the diagnostic signal lives in PC3. Think about this before reading on:

- If we add PC3, we get AUC = 0.53 — barely above chance. But adding PC4 jumps to 0.91, and all 6 PCs gives 0.92 (matching raw EMG exactly, as it must — 6 PCs is just a lossless rotation of the original 6 muscles).

The pattern is revealing: the diagnostic information is **concentrated in PC4** — a component that PCA ranks fourth because it captures relatively little total variance. With 2 PCs the signal is absent (AUC = 0.36), with 3 it barely appears (0.53), but the moment PC4 enters, AUC jumps to 0.91. This makes sense biologically. PCA finds the axes of largest overall variance, which in our data are dominated by directional differences (different muscles fire for different reach targets). The healthy-vs-impaired difference involves subtler shifts in co-activation ratios — a pattern that contributes little to total variance but is exactly what distinguishes the groups.

This is a general lesson, not just a quirk of our dataset: **variance explained is not the same as diagnostic relevance.** A dimensionality reduction that is optimal for summarizing data (PCA's goal) can be catastrophic for classification. Section 6 will confirm this with a direct comparison.

## 5. Can We Do Better?

Our first attempt with logistic regression gave us a baseline accuracy. Before trying fancier algorithms (Weeks 6–14), we can squeeze more out of this model by asking two questions:

### How many features should we use?

So far, every pipeline has used choices we made by hand: 2 PCA components (because Week 4 showed they capture most variance) and C = 1.0 (sklearn's default, introduced in Section 2). These are **hyperparameters** — settings chosen *before* training that affect performance. But the thought exercise after Figure 7 revealed that PC4 carries most of the diagnostic signal. Should we have used 4 components all along? And is C = 1.0 really the best regularization strength, or were we just lucky?

**GridSearchCV** automates this search. We give it a pipeline and a grid of values to try. Here, we use the 8-direction task (healthy subjects only, 240 trials) with this pipeline:

`StandardScaler` → `PCA(n_components=?)` → `LogisticRegression(C=?)`

The input is the same raw EMG matrix from Week 4: 240 trials × 6 muscles (peak envelope amplitudes). The scaler standardizes each muscle to zero mean and unit variance. PCA then reduces the 6 features to *n* components. Logistic regression classifies the PCA scores into one of 8 directions, with regularization strength C controlling how much the model is allowed to fit the training data.

We define a grid: n_components ∈ {1, 2, 3, 4, 5, 6} and C ∈ {0.01, 0.1, 1, 10, 100} — that is 6 × 5 = 30 combinations. For each combination, GridSearchCV fits the pipeline on 4 of the 5 folds, predicts on the held-out fold, and records accuracy (fraction of trials with correct direction labels). It repeats this 5 times (one per fold) and averages. The combination with the highest mean accuracy wins.

The **objective function** here is classification accuracy — the simplest metric. We could instead optimize AUC, F1-score, or any other metric by changing the `scoring` parameter. For the 8-direction task, accuracy is appropriate because all 8 classes matter equally. For the binary clinical task, we would prefer AUC (as in Figure 7) because it captures the sensitivity–specificity tradeoff that accuracy hides.

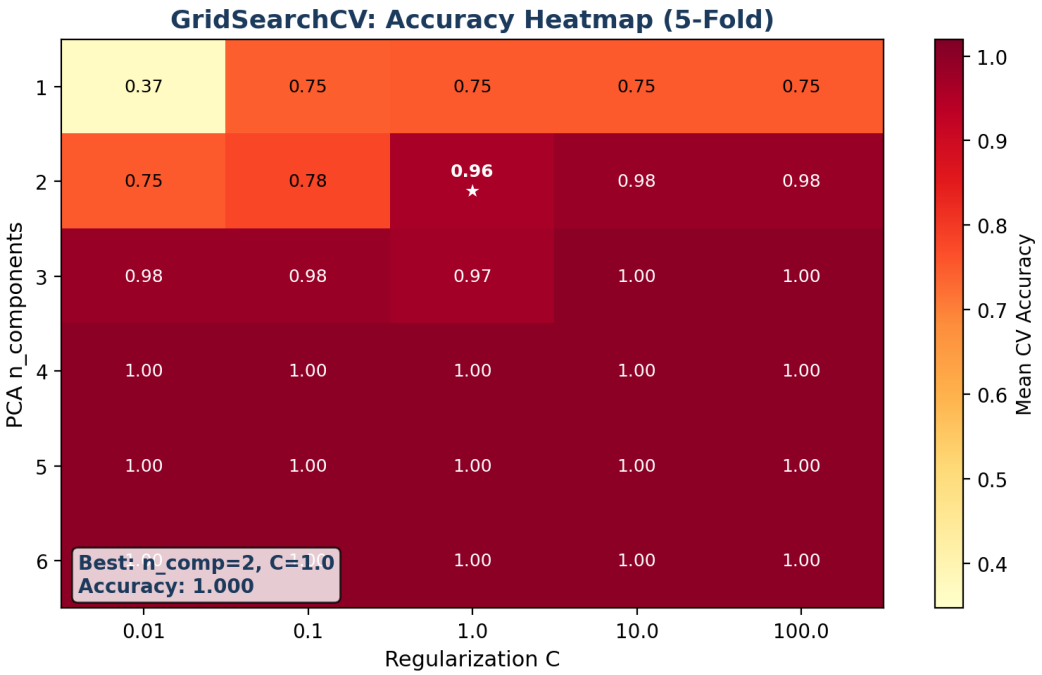


*Figure 8. GridSearchCV heatmap for 8-direction classification (healthy subjects, 5-fold CV). Each cell shows mean accuracy for one (n_components, C) combination. The star marks the best. Rows = number of PCA components; columns = regularization strength C.*

## Reading the heatmap

Start with the top row (n_components = 1). With a single PC, accuracy plateaus at 75% regardless of C — the model has only one feature and cannot separate 8 directions no matter how hard it tries. One dimension simply does not contain enough directional information.

Now read down the first column (C = 0.01, strong regularization). At n = 1, accuracy is just 37% — the model is so constrained it barely learns anything. But at n = 2, it jumps to 75%, and by n = 3 it reaches 98%. Even a heavily regularized model can succeed if it has enough features. Moving right along any row (increasing C, loosening regularization), accuracy either stays the same or improves slightly — the model has more freedom to fit the directional patterns.

The most striking pattern: from n = 4 onward, *every* cell is 1.00. This means that with 4 or more PCA components, logistic regression perfectly separates all 8 directions in healthy subjects regardless of regularization. The direction-decoding problem is easy once you have enough features — which is why our LOSO accuracy for this task was already high (80.8% on all subjects, 100% on healthy only).

When should you use GridSearchCV vs ROC analysis? They answer different questions. GridSearchCV is a **model selection** tool — use it to choose hyperparameters (how many features, how much regularization) before you deploy the model. ROC analysis is a **model evaluation** tool — use it after you have a trained model to understand how it trades sensitivity for specificity. In a typical workflow, you run GridSearchCV first to find the best configuration, then examine ROC curves of the winning model to decide how to set the classification threshold.

## Which muscles matter most for diagnosis?

A clinician looking at our model would ask a natural question: *which muscles are actually driving the diagnosis?* If the difference between healthy and impaired is concentrated in, say, the shoulder muscles, that tells us something about the underlying pathology — it points to proximal motor control deficits, altered synergy recruitment at the shoulder, or compensatory strategies. If all 6 muscles contribute equally, the impairment is more diffuse. Either answer shapes clinical interpretation and guides where to focus rehabilitation.

We can answer this by looking at our model's fitted weights. Recall from Section 2 that each muscle gets a weight — its contribution to the diagnostic decision. A large positive weight means higher activation in that muscle pushes the prediction toward 'impaired'; a large negative weight pushes toward 'healthy.' A weight near zero means the muscle is irrelevant for diagnosis.

But there is a problem: when we fit logistic regression with C = 1.0 on raw EMG, all 6 muscles get nonzero weights. Is each one truly contributing unique diagnostic information, or are some just picking up noise or redundancy? This is where **regularization** — the C parameter from Section 2 — becomes a diagnostic tool, not just a technical detail.

The idea is simple: we refit the model many times, each time with a different value of C, and watch what happens to the weights. With very small C (strong regularization), the model is forced to be parsimonious — it can only keep the muscles that contribute the most. As we increase C (loosen regularization), more muscles are allowed to enter. The **order in which muscles appear** reveals their diagnostic importance.

There are two flavors of regularization, and they answer different questions:

- **L1 (Lasso)** drives unimportant weights to *exactly zero*. As C increases, muscles enter one at a time. The first to appear are the most diagnostically important — this is automatic **feature selection**.
- **L2 (Ridge)** shrinks all weights toward zero but never eliminates them entirely. Every muscle always contributes something. This tends to give better prediction accuracy but does not identify a minimal set of biomarkers.

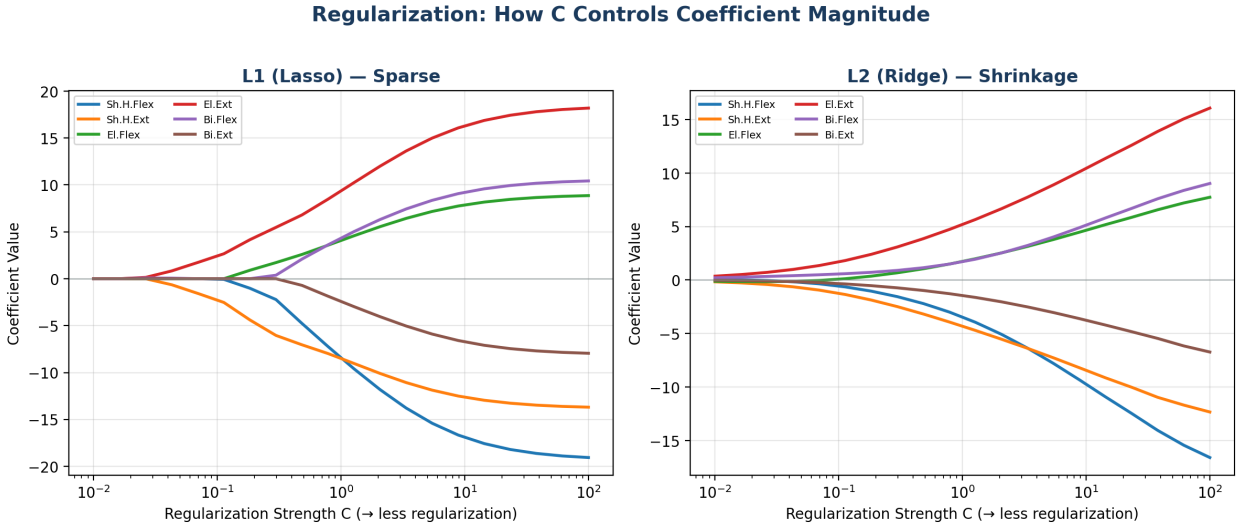

*Figure 9. Regularization paths for the healthy-vs-impaired task. Each line is one muscle's weight as C increases (left to right = less regularization). Left panel (L1): muscles enter sequentially — elbow extensors and shoulder horizontal extensors appear first, meaning they carry the strongest diagnostic signal. Biceps extensors enter last. Right panel (L2): all muscles grow simultaneously; no feature selection.*

Read the left panel (L1) from left to right. At very small C, all weights are zero — the model refuses to commit. The first muscles to break away from zero are **elbow extensors** and **shoulder horizontal extensors** (around C = 0.03). These are the muscles the model considers most essential for distinguishing groups. Next come **shoulder horizontal flexors** and **elbow flexors** (around C = 0.1–0.2). Last is **biceps extensors** (C ≈ 0.5), which the model considers least important.

This is a testable motor control finding. The model is telling us that the diagnostic difference between healthy and impaired subjects is most visible at the elbow and shoulder — the proximal joints where synergy merging (Week 4) has its largest effect. Distal muscles (biceps extensors) add relatively little diagnostic information. A clinician could use this to design a screening protocol: if you can only record from 2 muscles, choose elbow extensors and shoulder horizontal extensors.

Now compare with the right panel (L2). All 6 muscles grow together — no feature selection, no ranking. L2 gives you a better-performing model but cannot tell you which muscles matter most. The choice between L1 and L2 depends on your goal: L1 for clinical insight (which muscles?), L2 for maximum prediction accuracy.

> The regularization path is a bridge between machine learning and motor control. It transforms the abstract question 'which features matter?' into the concrete clinical answer 'the diagnostic difference is concentrated in the proximal extensors.'

## 6. Does the Feature Representation Matter?

In Week 4, we spent considerable effort extracting motor synergies from the EMG data — running PCA to find the dominant patterns of muscle co-activation, then applying Varimax rotation to make those synergies more physiologically interpretable. The synergy framework is appealing because it reduces 6 muscle channels to 2–3 synergy activations, and these synergies have clear motor control meaning: one captures a push/pull pattern, another a proximal/distal pattern.

But a compact, interpretable representation is only useful for classification if it preserves the information the classifier needs. Figures 5 and 7 already showed that 2 PCs lose the diagnostic signal entirely. Here we ask: does adding a third PC help? And does Varimax rotation — which rearranges the same variance into more interpretable axes — rescue any diagnostic information?

We compare three inputs to the same logistic regression, all evaluated with LOSO:

- **Raw EMG:** all 6 peak muscle amplitudes, standardized. No compression.
- **PCA scores (3 components):** projections onto the top 3 principal components, capturing roughly 85% of total variance.
- **Varimax scores (3 components):** the same 3 PCs after Varimax rotation. Same variance explained, but axes realigned to load cleanly onto muscle groups.

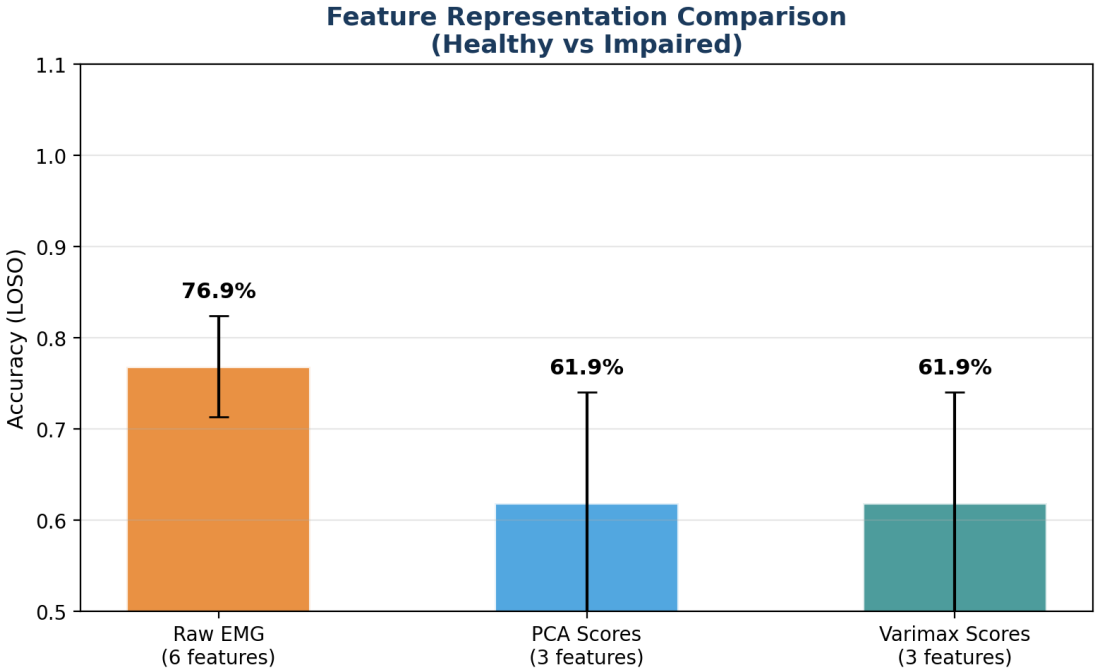


*Figure 10. Accuracy comparison across three feature representations for healthy vs impaired (LOSO). Raw EMG substantially outperforms PCA and Varimax scores, confirming that compression to 3 components loses diagnostic information — consistent with the dual-pipeline results in Figures 5 and 7.*

The result is clear: raw EMG (77%) substantially outperforms both PCA and Varimax (62% each). Three observations are worth highlighting:

First, PCA and Varimax give **identical accuracy**. This makes mathematical sense — Varimax is just a rotation of the PCA axes, so it spans the same subspace. Rotation changes which muscles load onto which component (improving interpretability) but does not add or remove any information. What PCA cannot classify, Varimax cannot either.

Second, even with 3 components (up from 2), the compressed representations still lose substantial diagnostic information. This is consistent with the thought exercise after Figure 7: the diagnostic signal is concentrated in PC4, so 3 components are still not enough. The synergy framework from Week 4 is excellent for understanding the *structure* of motor coordination — which muscles co-activate and how those patterns differ between groups — but it is not the right input for a clinical classifier.

Third, this creates a useful tension for motor control research. Synergies are interpretable and compact. Raw EMG is high-dimensional and harder to explain. The classifier needs raw EMG, but the clinician needs synergy language. In later weeks we will explore methods that can bridge this gap — models that classify well *and* reveal which synergy-level features drive the decision.

## How much data do we need?

Clinical motor control studies face a constant tension: impaired subjects are hard to recruit. A stroke study might spend months enrolling 20 patients. If our classifier needs 50 subjects to work reliably, that is a practical barrier. If 10 suffice, the tool becomes clinically feasible. The **learning curve** answers this question by training the model on increasing amounts of data and tracking how accuracy changes.

A note on what the x-axis shows: Figure 11 uses **5-fold CV** (not LOSO) and plots the number of training *trials*, not subjects. With 480 total trials (20 subjects × 24 trials each) and 5-fold CV, the maximum training set is 384 trials — roughly 16 subjects' worth of data. We use 5-fold here rather than LOSO because the learning curve requires varying the training set size, which is difficult with LOSO's fixed leave-one-out structure. The tradeoff is that trial-level splitting inflates accuracy slightly (as we saw in Figure 5), so these numbers are optimistic. The *shape* of the curve — whether it has plateaued or is still climbing — is more informative than the absolute accuracy values.

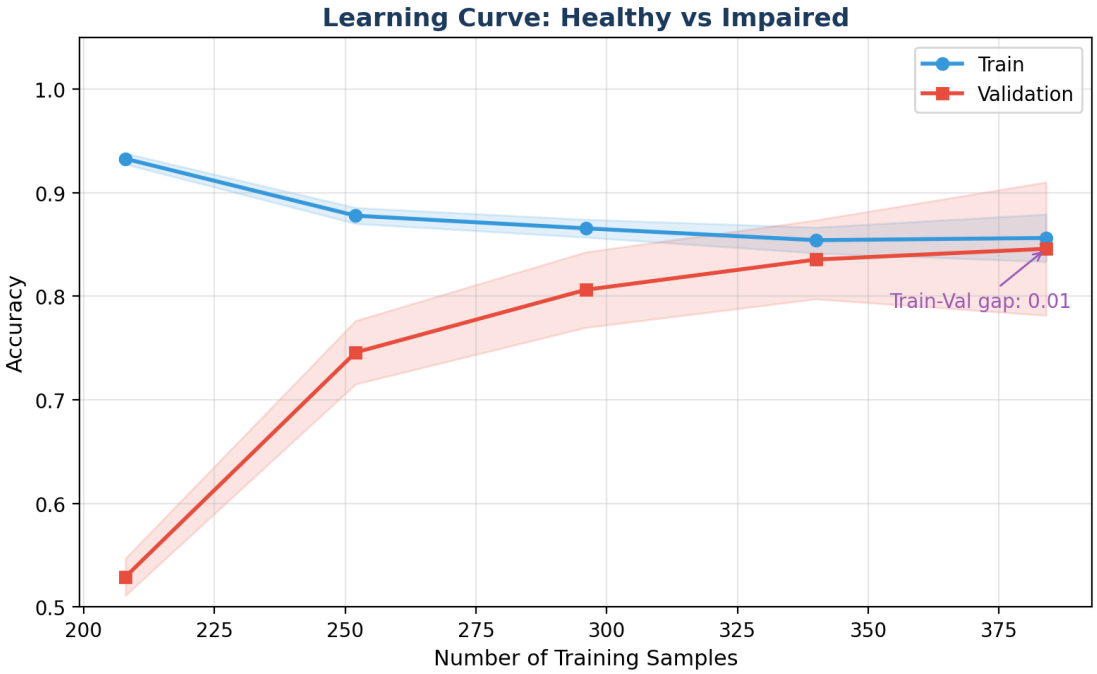


*Figure 11. Learning curve for healthy vs impaired (raw EMG, 5-fold CV). The x-axis shows training trials; dividing by 24 gives approximate subject count. Validation accuracy climbs steeply from ~53% (3 subjects) to ~85% (16 subjects), nearly meeting training accuracy — the model is not badly overfitting.*

Two things stand out. First, the steep early rise: going from ~3 to ~8 subjects' worth of data (76 to 208 trials) produces the largest accuracy gains. This suggests that even a modest pilot study could build a useful initial classifier. Second, the curve has not fully plateaued at our maximum of ~16 subjects — validation accuracy is still creeping upward. More subjects would likely improve performance further, which is relevant for planning future data collection.

For a clinical deployment decision, the key question is where the curve bends: if you are designing a multi-site study, the learning curve tells you how many subjects per site you need before the classifier stabilizes. In our case, ~10 subjects (240 trials) gets you most of the way, but 20+ would be safer for a robust screening tool.

# 7. The Payoff: How Close Did We Get?

We started this lecture with a question: if you had only the EMG data and no clinical labels, how close could you get to the clinician's diagnosis?

Here is the answer. We took our best logistic regression model — raw EMG features (all 6 muscles), regularization tuned to C = 10 via GridSearchCV, tested with LOSO so that every prediction comes from a model that never saw that subject during training — and compared its predictions to the clinician's labels, trial by trial:

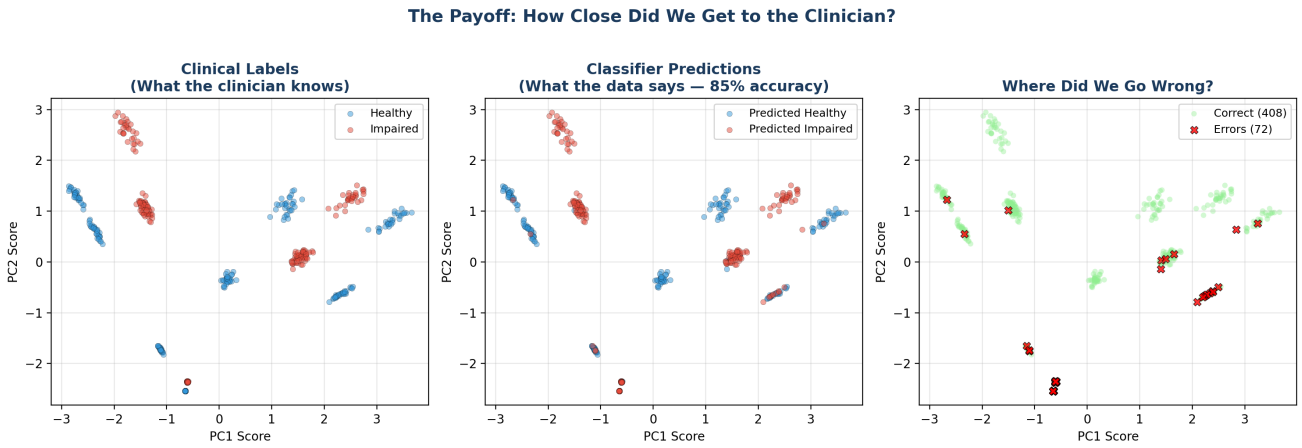


*Figure 12. The payoff. Left: the clinician's labels (ground truth). Center: the classifier's predictions from raw EMG alone (85% accuracy). Right: where the classifier went wrong (red X's). Points are plotted in PCA space for visualization, but the classifier used all 6 raw muscle amplitudes — not PCA scores — as input.*

The raw EMG classifier achieves **85% accuracy** (LOSO) with an **AUC of 0.94**. These are the numbers for our tuned model (C = 10). For comparison, the default model (C = 1, reported in earlier sections) achieved 77% accuracy and AUC = 0.92 — tuning the regularization strength gained us 8 percentage points in accuracy.

An important subtlety in Figure 12: the trials are *plotted* in PCA space (PC1 vs PC2) because that is the visualization we have been using since Week 4. But the classifier that generated these predictions did **not** use PCA scores — it used all 6 raw muscle amplitudes. This is why some errors appear in regions where the two groups look well-separated in the plot: the 2D PCA projection hides the dimensions where the confusion actually occurs. The classifier operates in the full 6-dimensional muscle space, where the boundary is more complex than any 2D view can show.

Where does the classifier struggle? The errors cluster near the boundary between groups — the clinically ambiguous cases where even the muscle patterns overlap. These are subjects whose synergy structure falls between the typical healthy and impaired patterns. From a motor control perspective, these borderline cases are the most interesting: they may represent early-stage impairment, compensatory strategies that partially mask deficits, or simply natural variation in how muscles coordinate. Understanding *why* the classifier fails on specific trials is itself a research question — one that links machine learning back to motor neuroscience.

## Looking ahead

Logistic regression is our baseline — the simplest possible classifier. The coming weeks build on it in two ways:

- **Week 6 adds new data, not a new algorithm.** We extend the reaching model with a simulated motor cortex population (~80 neurons). Logistic regression on neural features will fill in new columns alongside the EMG columns below — same method, different inputs. Do neurons carry more diagnostic information than muscles?
- **Weeks 7–11 add new algorithms.** Each week (Naive Bayes, KNN, SVM, Random Forests, LDA) fills in a new row using the **same raw EMG features**, the **same LOSO procedure**, and compared against the **same clinician labels**. Every method also gets evaluated on neural features for a dual comparison.

**Running Methods Comparison Table (Updated Each Week)**

| | 8-Dir Acc (LOSO) | H vs I Acc (LOSO) | AUC-ROC | Interpret-ability | Status |
|---|---|---|---|---|---|
| **Logistic Reg (Week 5)** | 80.8% | 85.0% | AUC=0.94 | Medium | ⬅ Baseline |
| **GLM (Week 6)** | ? | ? | ? | ? | Week 6 |
| **Naive Bayes (Week 7)** | ? | ? | ? | ? | Week 7 |
| **KNN (Week 8)** | ? | ? | ? | ? | Week 8 |
| **SVM (Week 9)** | ? | ? | ? | ? | Week 9 |
| **Random Forest (Week 10)** | ? | ? | ? | ? | Week 10 |
| **LDA (Week 11)** | ? | ? | ? | ? | Week 11 |

*Figure 13. Running methods comparison. All metrics use raw EMG features with LOSO cross-validation, ensuring an apples-to-apples comparison across methods. Logistic regression (C = 10) sets the baseline: 80.8% direction accuracy, 85.0% clinical accuracy, AUC = 0.94. Week 6 adds neural-feature columns; Weeks 7–11 add algorithm rows.*

Logistic regression is our baseline. Every future method must earn its added complexity by getting closer to the clinician's labels than this simple line. And in Week 6, we ask: does richer data (neurons) help more than a fancier algorithm?